Дашборды компании

ПОКАЗАТЕЛИ СЛУЖБ АЛМИ ПАРТНЕР

Руководство по развёртыванию

Версия 0.2.0

Ноябрь 2026

© Alexander Yurasov

Содержание

1 О документе 3

2 Архитектура 4

3 Требования 5

3.1 Сервер 5

3.2 Клиенты 5

3.3 Сборка из исходного кода 5

4 Установка 6

4.1 Шаг 1. Получение дистрибутива 6

4.2 Шаг 2. Конфигурация 6

4.3 Шаг 3. Сборка и запуск 6

4.4 Шаг 4. Проверка 7

4.5 Шаг 5. HTTPS 7

4.6 Шаг 6. Первый вход 8

5 Демонстрационный стенд 9

6 Переменные окружения 10

6.1 Основные 10

6.2 Сессии и безопасность 10

6.3 Вход по домену (LDAP) 11

6.4 Синхронизация с Active Directory 13

6.5 Производительность 14

6.6 Журнал 15

7 Вход по домену и Active Directory 16

7.1 Настройка входа по домену 16

7.2 Синхронизация по расписанию 17

8 База данных и миграции 20

9 Резервное копирование и восстановление 21

9.1 PostgreSQL 21

9.2 SQLite 21

10 Обновление 22

10.1 Обычное обновление 22

10.2 Обновление до 0.2.0 22

10.3 Обновление до 0.1.0 с версий до 0.1.0 24

11 Производительность и масштабирование 25

12 Обслуживание 26

13 Диагностика 27

1 О документе

Руководство описывает установку, настройку, обновление и обслуживание «Дашбордов компании» версии 0.2.0: состав системы, требования к серверу, запуск в Docker, переменные окружения (в том числе часовой пояс, вход по учётной записи домена и синхронизацию с Active Directory), миграции базы данных, резервное копирование, обновление до 0.2.0, проверку производительности и диагностику.

Документ предназначен для системных администраторов и инженеров эксплуатации. Работа с интерфейсом описана в «Руководстве пользователя», администрирование учётных записей, доступа, структуры компании и служб — в «Руководстве администратора».

Условные обозначения

ОБОЗНАЧЕНИЕ	ЗНАЧЕНИЕ
`команда`	Команда, имя файла или переменной окружения
> **Важно.**	Правило, нарушение которого приводит к ошибке или потере данных
> **Совет.**	Рекомендация

2 Архитектура

Система состоит из двух компонентов, которые поставляются одним Docker-образом, и базы данных; каталог домена подключается по желанию:

КОМПОНЕНТ	ТЕХНОЛОГИИ	НАЗНАЧЕНИЕ
Интерфейс	Vue 3, Vite, Pinia, Vue Router, Apache ECharts	Одностраничное приложение; собирается при сборке образа и раздаётся сервером
Сервер	Python 3.12, FastAPI, SQLAlchemy 2, Alembic, gunicorn + uvicorn	API, проверка прав, расчёт аналитики, структура компании, PDF-отчёты (ReportLab, Matplotlib), импорт и экспорт Excel (openpyxl), вход по домену и синхронизация с AD (ldap3)
База данных	PostgreSQL 16 (рекомендуется) или SQLite	Данные служб, структура компании, пользователи и доступ, сессии, журнал изменений
Каталог (необязательно)	Active Directory, FreeIPA, OpenLDAP	Вход по учётной записи домена, синхронизация подразделений и сотрудников

Запросы обслуживают несколько рабочих процессов gunicorn с воркерами uvicorn. Состояние хранится только в базе данных (включая сессии), поэтому процессы и экземпляры приложения взаимозаменяемы: их число можно увеличивать без настройки «липких» сессий. Снимок структуры компании каждый процесс держит в памяти и перестраивает только при изменении структуры (по её ревизии в базе).



3 Требования

3.1 Сервер

ПАРАМЕТР	МИНИМУМ	РЕКОМЕНДУЕТСЯ ДЛЯ 500–1000 ПОЛЬЗОВАТЕЛЕЙ
Процессор	2 ядра	4 ядра и более
Память	2 ГБ	4–8 ГБ
Диск	10 ГБ	20 ГБ SSD
ОС	Linux x86_64 с Docker 24+ и Docker Compose v2	Ubuntu 24.04 LTS / Astra Linux / РЕД ОС

Отдельный сервер PostgreSQL не обязателен: `docker-compose.yml` запускает его в соседнем контейнере. Для промышленной эксплуатации можно использовать внешний кластер PostgreSQL 14+.

Для входа по домену и синхронизации с Active Directory серверу приложения нужен доступ к контроллеру домена по LDAPS (порт 636) или LDAP со StartTLS (порт 389), служебная учётная запись с правом чтения каталога и сертификат центра сертификации домена, если он не доверенный в системе.

3.2 Клиенты

Современный браузер: Яндекс Браузер, Chrome, Edge, Firefox последних версий; экран от 1280 пикселей по ширине (на узких экранах и телефонах интерфейс перестраивается).

3.3 Сборка из исходного кода

Для разработки и сборки без Docker нужны Python 3.12+, Node.js 20.19+ (рекомендуется 22) и npm.

4 Установка

4.1 Шаг 1. Получение дистрибутива

```
git clone https://github.com/ayurasov/dashboards_serv.git
cd dashboards_serv
git checkout v0.2.0
```

4.2 Шаг 2. Конфигурация

```
cp .env.example .env
```

Откройте `.env` и задайте как минимум:

ПЕРЕМЕННАЯ	ЧТО УКАЗАТЬ
`SECRET_KEY`	Случайная строка не короче 32 символов: `openssl rand -hex 32`
`POSTGRES_PASSWORD`	Пароль пользователя базы данных
`ADMIN_PASSWORD`	Пароль первого администратора (не короче 8 символов). Если оставить пустым, временный пароль будет выведен в журнал контейнера
`APP_PORT`	Порт приложения на сервере (по умолчанию 8000)
`TIMEZONE`	Часовой пояс компании, если он не московский (по умолчанию `Europe/Moscow`)

Вход по учётной записи домена настраивается позже, после первого входа (раздел «Вход по домену и Active Directory»).

> **Важно.** С ключом по умолчанию сервер в режиме production не запускается. Храните `.env` вне системы контроля версий.

4.3 Шаг 3. Сборка и запуск

```
docker compose up -d --build
docker compose logs -f app
```

При старте контейнер приложения:

1. ждёт готовности PostgreSQL;
2. применяет миграции базы данных (`python -m app.cli migrate`) — под блокировкой, поэтому одновременный старт нескольких контейнеров безопасен;
3. приводит справочники к актуальному виду (службы, метрики, правила светофора, палитры, встроенные профили доступа, типы уровней структуры);
4. создаёт первого администратора, если пользователей в базе нет;
5. запускает gunicorn.

4.4 Шаг 4. Проверка

```
curl -s http://127.0.0.1:8000/api/health
```

Ответ `{"status": "ok", "version": "0.2.0", "database": "ok", "schema": {"current": "0014_org_access", "head": "0014_org_access"}}` означает, что сервер работает, база доступна и схема актуальна. Статус `degraded` (код 503) — схема базы не совпадает с версией приложения: проверьте журнал миграций.

Откройте `http://<сервер>:8000` и войдите под учётной записью администратора. Документация API (OpenAPI) доступна по адресу `/api/docs`.

4.5 Шаг 5. HTTPS

Приложение рассчитано на работу за обратным прокси, который завершает TLS. Пример для Caddy (сертификат выпускается автоматически):

```
dashboards.company.ru {
    encode gzip
    reverse_proxy 127.0.0.1:8000
}
```

Пример для nginx:

```
server {
    listen 443 ssl http2;
    server_name dashboards.company.ru;
    ssl_certificate     /etc/ssl/dashboards.crt;
    ssl_certificate_key /etc/ssl/dashboards.key;
    client_max_body_size 25m;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 120s;
    }
}
```

После включения HTTPS задайте `SESSION_COOKIE_SECURE=true` и перезапустите приложение: cookie сессии будет передаваться только по защищённому соединению.

4.6 Шаг 6. Первый вход

1. Войдите под логином из `ADMIN_USERNAME` (по умолчанию `admin`) и паролем из `ADMIN_PASSWORD` или временным паролем из журнала (`docker compose logs app | grep администратор`).
2. Система попросит сменить пароль.
3. Постройте структуру компании и создайте учётные записи сотрудников с профилями доступа («Руководство администратора», разделы «Структура компании и подразделения», «Пользователи», «Импорт и экспорт»).
4. При необходимости включите вход по домену и синхронизацию с Active Directory (раздел «Вход по домену и Active Directory»).
5. Загрузите данные служб: недельный отчёт техподдержки и выгрузку Naumen — из файлов Excel, данные службы персонала — вводом по месяцам.

5 Демонстрационный стенд

Для показа и обучения можно запустить систему с демонстрационными данными: структура компании из 13 подразделений, девять учётных записей разных уровней (от генерального директора до инженера) с профилями доступа, семь месяцев данных службы персонала, 109 недель отчёта техподдержки, 1868 заявок Naumen и 142 партнёрства.

```
DEMO_DATA=true docker compose up -d --build
```

Демонстрационные данные загружаются только в пустые разделы и никогда не перезаписывают существующие. Логины и пароли демо-стенда приведены в «Руководстве администратора» (раздел «Демонстрационный стенд»).

> **Важно.** Не включайте `DEMO_DATA` на рабочем сервере: демо-пользователи получают известные пароли.

6 Переменные окружения

`docker-compose.yml` передаёт контейнеру приложения переменные из таблиц «Основные», «Сессии и безопасность» и основные переменные входа по домену (`AUTH_LDAP_ENABLED` , `AUTH_LDAP_URL` , `AUTH_LDAP_BASE_DN` , `AUTH_LDAP_BIND_DN` , `AUTH_LDAP_BIND_PASSWORD`). Остальные переменные добавьте в раздел `environment` сервиса `app` — удобнее в файле `docker-compose.override.yml` , который Docker Compose подключает автоматически (пример — в разделе «Вход по домену и Active Directory»).

6.1 Основные

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`APP_ENV`	`production` в образе	`production` — строгие проверки конфигурации; `development` — для разработки
`DATABASE_URL`	задаёт compose	Адрес базы: `postgresql+psycopg://user:pass@host:5432/db` или `sqlite:////data/dashboards.db`
`SECRET_KEY`	—	Ключ подписи, не короче 32 символов
`ADMIN_USERNAME`	`admin`	Логин первого администратора
`ADMIN_PASSWORD`	пусто	Пароль первого администратора; пусто — временный пароль в журнале
`DEMO_DATA`	`false`	Загрузить демонстрационные данные в пустые разделы
`TIMEZONE`	`Europe/Moscow`	Часовой пояс компании (имя из базы IANA, например `Asia/Yekaterinburg`): границы дней в фильтре журнала изменений, время формирования в PDF-отчётах, даты, зависящие от «сегодня». Неизвестное имя — UTC
`CORS_ORIGINS`	пусто	Разрешённые источники через запятую — только если интерфейс открывается с другого адреса

6.2 Сессии и безопасность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`SESSION_LIFETIME_HOURS`	`12`	Максимальная длительность сессии

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`SESSION_IDLE_MINUTES`	`240`	Сессия завершается после простоя
`SESSION_COOKIE_SECURE`	`false`	`true` — cookie только по HTTPS
`PASSWORD_MIN_LENGTH`	`8`	Минимальная длина пароля
`LOGIN_RATE_LIMIT`	`10`	Неудачных попыток входа с одного адреса для одного логина за окно
`LOGIN_RATE_WINDOW_SECONDS`	`300`	Окно ограничения попыток входа

6.3 Вход по домену (LDAP)

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AUTH_LDAP_ENABLED`	`false`	`true` — включить вход по учётной записи домена
`AUTH_LDAP_URL`	пусто	Адрес контроллера домена: `ldaps://dc.company.local:636` (рекомендуется) или `ldap://dc.company.local:389` со StartTLS
`AUTH_LDAP_BASE_DN`	пусто	База поиска учётных записей, например `DC=company,DC=local`
`AUTH_LDAP_BIND_DN`	пусто	Служебная учётная запись только для чтения; пусто — анонимная привязка
`AUTH_LDAP_BIND_PASSWORD`	пусто	Пароль служебной учётной записи (в API и журнал не попадает)
`AUTH_LDAP_USER_FILTER`	`(&(objectClass=person)(|(sAMAccountName={login})(userPrincipalName={login})(mail={login})(uid={login})))`	Фильтр поиска; `{login}` — введённый логин (экранируется)

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AUTH_LDAP_ATTR_ID`	`objectGUID`	Неизменный идентификатор записи; FreeIPA и OpenLDAP — `entryUUID`
`AUTH_LDAP_ATTR_EMAIL`	`mail`	Атрибут e-mail — по нему учётные записи сопоставляются
`AUTH_LDAP_ATTR_NAME`	`displayName`	ФИО; FreeIPA — `cn`
`AUTH_LDAP_ATTR_LOGIN`	`sAMAccountName`	Логин для созданных учётных записей; FreeIPA — `uid`
`AUTH_LDAP_ATTR_POSITION`	`title`	Должность (заполняет пустую должность в системе)
`AUTH_LDAP_ATTR_GROUPS`	`memberOf`	Группы — для профиля доступа и группы администраторов
`AUTH_LDAP_CA_FILE`	пусто	Файл сертификата центра сертификации домена (PEM) внутри контейнера; для `ldap://` включает StartTLS
`AUTH_LDAP_START_TLS`	`false`	StartTLS для `ldap://` (для `ldaps://` не нужен)
`AUTH_LDAP_TIMEOUT`	`5`	Ожидание ответа каталога, с (1–30)
`AUTH_LDAP_CREATE_USERS`	пусто	Первый вход без учётной записи: пусто — отказ, `active` — создать активной, `inactive` — создать отключённой
`AUTH_LDAP_DEFAULT_PROFILE`	пусто	Ключ профиля доступа для созданных при входе (например, `employee`)
`AUTH_LDAP_GROUP_PROFILES`	пусто	Профили по группам: `DN группы=ключ профиля` через `;`, первое совпадение
`AUTH_LDAP_ADMIN_GROUP`	пусто	DN группы, участники которой создаются администраторами

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AUTH_LDAP_LOGIN_HINT`	пусто	Подсказка на странице входа; пусто — «Можно войти с логином и паролем домена — как при входе в Windows»

Сертификат сервера каталога проверяется всегда (своим файлом `AUTH_LDAP_CA_FILE` или системными корневыми сертификатами). Без `ldaps://` и StartTLS пароли идут по сети открытым текстом — страница «Вход по домену» предупредит об этом.

6.4 Синхронизация с Active Directory

Пустые адрес, база поиска, служебная учётная запись, сертификат и StartTLS берутся из настроек входа по домену; включать сам вход по домену (`AUTH_LDAP_ENABLED`) для синхронизации не обязательно.

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AD_SYNC_URL`	пусто → `AUTH_LDAP_URL`	Адрес каталога для синхронизации
`AD_SYNC_BASE_DN`	пусто → `AUTH_LDAP_BASE_DN`	База поиска сотрудников
`AD_SYNC_BIND_DN`	пусто → `AUTH_LDAP_BIND_DN`	Служебная учётная запись (берётся парой с паролем: своя или та же, что у входа)
`AD_SYNC_BIND_PASSWORD`	пусто → `AUTH_LDAP_BIND_PASSWORD`	Пароль служебной учётной записи
`AD_SYNC_USER_FILTER`	`(&(objectCategory=person)(objectClass=user)(mail=*))`	Какие записи считаются сотрудниками
`AD_SYNC_PATH_SOURCE`	`attributes`	Путь подразделения: `attributes` — из атрибутов, `ou` — из организационных единиц в DN
`AD_SYNC_PATH_ATTRIBUTES`	`company,division,department`	Атрибуты пути подразделения сверху вниз
`AD_SYNC_DETECT_HEADS`	`true`	Определять руководителей подразделений по атрибуту `manager` подчинённых
`AD_SYNC_CA_FILE`	пусто → `AUTH_LDAP_CA_FILE`	Сертификат центра сертификации

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`AD_SYNC_START_TLS`	`false`	StartTLS для `ldap://`; без своего `AD_SYNC_URL` действует `AUTH_LDAP_START_TLS`

Ожидание ответа — `AUTH_LDAP_TIMEOUT`. Каталог читается постранично.

6.5 Производительность

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`WEB_CONCURRENCY`	2 × ядра + 1 (не больше 9)	Число рабочих процессов
`DB_POOL_SIZE`	`5`	Постоянных соединений с базой на процесс
`DB_MAX_OVERFLOW`	`4`	Дополнительных соединений на процесс при пике
`DB_POOL_TIMEOUT`	`30`	Ожидание свободного соединения, с
`DB_CONCURRENCY`	`0` — по числу соединений	Запросов, одновременно работающих с базой, в процессе; остальные ждут в очереди
`DB_QUEUE_TIMEOUT`	`30`	Ожидание в очереди к базе, с; дольше — ответ 503 «Сервер перегружен»
`THREADPOOL_SIZE`	`0` — соединения + 8	Потоков для обработчиков в процессе
`PDF_CONCURRENCY`	`2`	PDF-отчётов, одновременно строящихся в процессе
`RESPONSE_CACHE`	`true`	Кэш ответов аналитики в памяти процесса
`RESPONSE_CACHE_SIZE`	`512`	Ответов в кэше процесса
`GUNICORN_TIMEOUT`	`120`	Тайм-аут запроса, с
`GUNICORN_MAX_REQUESTS`	`2000`	Перезапуск процесса после N запросов (страховка от роста памяти)

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`POSTGRES_MAX_CONNECTIONS`	`200`	`max_connections` контейнера PostgreSQL
`MAX_UPLOAD_MB`	`20`	Максимальный размер загружаемого файла

Совет. Соединений с базой нужно не больше `WEB_CONCURRENCY × (DB_POOL_SIZE + DB_MAX_OVERFLOW)` на экземпляр приложения — проверьте, что `max_connections` PostgreSQL с запасом это покрывает. Значения по умолчанию (9 × 9 = 81) укладываются даже в стандартные `max_connections` = 100.

6.6 Журнал

ПЕРЕМЕННАЯ	ПО УМОЛЧАНИЮ	НАЗНАЧЕНИЕ
`LOG_LEVEL`	`INFO`	Уровень журнала
`LOG_JSON`	`false`	Журнал в формате JSON (для Loki, ELK и т. п.); пароли, токены и сессии в журнал не попадают

7 Вход по домену и Active Directory

7.1 Настройка входа по домену

1. Заведите в каталоге служебную учётную запись только для чтения и узнайте базу поиска (`DC=company,DC=local`).
2. Если сертификат контроллера домена выпущен внутренним центром сертификации, положите сертификат центра (PEM) на сервер, например в `/opt/dashboards_serv/certs/company-ca.pem`.
3. Задайте основные переменные в `.env` — сначала с выключенным входом:

```
AUTH_LDAP_ENABLED=false
AUTH_LDAP_URL=ldaps://dc.company.local:636
AUTH_LDAP_BASE_DN=DC=company,DC=local
AUTH_LDAP_BIND_DN=CN=svc-dashboards,OU=Service,DC=company,DC=local
AUTH_LDAP_BIND_PASSWORD=…
```

4. Остальные переменные и сертификат передайте через `docker-compose.override.yml` рядом с `docker-compose.yml`:

```
services:
  app:
    environment:
      AUTH_LDAP_CA_FILE: /certs/company-ca.pem
      AUTH_LDAP_CREATE_USERS: inactive
      AUTH_LDAP_DEFAULT_PROFILE: employee
      AUTH_LDAP_GROUP_PROFILES: "CN=Top
Management,OU=Groups,DC=company,DC=local=executive;CN=Directors,OU=Groups,DC=company,DC=l
ocal=director"
      AUTH_LDAP_ADMIN_GROUP: "CN=Dashboards Admins,OU=Groups,DC=company,DC=local"
      AUTH_LDAP_LOGIN_HINT: "Логин и пароль, как при входе в Windows"
      AD_SYNC_PATH_SOURCE: attributes
    volumes:
      - ./certs:/certs:ro
```

5. `docker compose up -d` и откройте **Администрирование → Вход по домену**: состояние «Настроен, но выключен», замечаний нет, шифрование — LDAPS или StartTLS. Нажмите **Проверить подключение** и найдите несколько сотрудников (**Найти учётную запись**) — каждый должен находиться ровно одной записью и сопоставляться с нужной учётной записью системы.
6. Задайте `AUTH_LDAP_ENABLED=true` и выполните `docker compose up -d`. Проверьте вход тестовым сотрудником.

Подробности о сопоставлении учётных записей, создании при первом входе и профилях по группам — «Руководство администратора», раздел «Вход по домену (LDAP)»; регламент — `docs/runbooks/upgrade-to-0.2.0.md`.

> **Важно.** Учётные записи сопоставляются с каталогом по e-mail. Держите e-mail локальных администраторов вне каталога (или пустым) и сохраните хотя бы одного администратора с паролем системы — он войдёт, даже если каталог недоступен.

7.2 Синхронизация по расписанию

Синхронизация с Active Directory запускается из интерфейса (**Импорт и экспорт → Active Directory**) или командой:

```
docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad                # только план
docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad --apply        # применить
```

ПАРАМЕТР	НАЗНАЧЕНИЕ
`--ad`	Читать сотрудников из Active Directory (`AD_SYNC_*`)
`--csv ФАЙЛ`	Вместо каталога — загрузка списком из файла `.csv` или `.xlsx` (путь внутри контейнера)
`--apply`	Применить план; без него команда только показывает план
`--create-missing`	Создавать сотрудников, которых ещё нет (из AD — со входом по домену)
`--deactivate-missing`	Только AD: отключить пришедших из AD, которых нет в каталоге
`--actor ЛОГИН`	От чьего имени записывать журнал изменений
`--json`	Вывести полный план в JSON

Коды выхода:

КОД	ЗНАЧЕНИЕ
`0`	Готово: план без ошибок (с `--apply` — применён)

КОД	ЗНАЧЕНИЕ
`1`	В плане есть ошибки — ничего не записано; список ошибок — в выводе команды
`2`	Каталог или файл недоступен, не задана настройка или не найден пользователь `--actor`

Пример для cron — каждую ночь в 03:15, вывод — в файл журнала:

```
# /etc/cron.d/dashboards-org-sync
15 3 * * * root cd /opt/dashboards_serv && docker compose exec -T -w /app/backend app python
-m app.cli org-sync --ad --apply --create-missing --actor admin >> /var/log/dashboards-org-
sync.log 2>&1
```

Пример для systemd: служба и таймер.

```
# /etc/systemd/system/dashboards-org-sync.service
[Unit]
Description=Дашборды компании: синхронизация структуры с Active Directory
After=docker.service
Requires=docker.service

[Service]
Type=oneshot
WorkingDirectory=/opt/dashboards_serv
ExecStart=/usr/bin/docker compose exec -T -w /app/backend app python -m app.cli org-sync --
ad --apply --create-missing --actor admin
```

```
# /etc/systemd/system/dashboards-org-sync.timer
[Unit]
Description=Синхронизация с Active Directory каждую ночь

[Timer]
OnCalendar=*-*-* 03:15
Persistent=true

[Install]
WantedBy=timers.target
```

```
systemctl daemon-reload
systemctl enable --now dashboards-org-sync.timer
systemctl list-timers dashboards-org-sync.timer     # когда следующий запуск
journalctl -u dashboards-org-sync.service -n 50     # вывод последнего запуска
```

Код выхода 1 или 2 переводит службу systemd в состояние `failed` (видно в `systemctl --failed`) — подключите его к мониторингу. Перед первым запуском по расписанию посмотрите план без `--apply`, а `--deactivate-missing` включайте, когда убедитесь, что фильтр

`AD_SYNC_USER_FILTER` выбирает всех нужных сотрудников.

8 База данных и миграции

Схемой базы управляет только Alembic: приложение само таблицы не создаёт и не меняет. Миграции применяются автоматически при старте контейнера; вручную:

```
docker compose exec -w /app/backend app python -m app.cli migrate        # применить миграции и обновить справочники
docker compose exec -w /app/backend app python -m app.cli check-schema   # код выхода 0 — схема актуальна
```

> **Важно.** Команды `python -m app.cli` выполняются в каталоге `/app/backend` контейнера — отсюда параметр `-w /app/backend`. Без него команда завершится ошибкой «No module named app».

PostgreSQL рекомендуется для всех установок. SQLite подходит для разработки и небольших стендов: в образе она работает в режиме WAL с проверкой внешних ключей.

МИГРАЦИЯ	ВЕРСИЯ	ЧТО ДЕЛАЕТ
`0012_v0_1_0_schema`	0.1.0	Перестройка всех таблиц с переносом данных прежних версий (откат не поддерживается)
`0013_data_revisions`	0.1.0	Ревизии данных служб для кэша ответов
`0014_org_access`	0.2.0	Структура компании, вход по домену, модель доступа 2.0 (откат поддерживается)

Список таблиц и связей — в «Руководстве администратора» (раздел «Данные и хранение») и в `docs/architecture/adr-0002-database-and-migrations.md`, `adr-0007-org-structure.md`, `adr-0009-access-model.md`.

9 Резервное копирование и восстановление

9.1 PostgreSQL

Резервная копия (без остановки приложения):

```
docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > dashboards-$(date
+%F).dump
```

Восстановление:

```
docker compose stop app
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists <
dashboards-2026-11-01.dump
docker compose start app
```

9.2 SQLite

```
docker compose exec app python -c "import sqlite3; s=sqlite3.connect('/data/dashboards.db');
d=sqlite3.connect('/data/backup.db'); s.backup(d)"
docker compose cp app:/data/backup.db ./dashboards-$(date +%F).db
```

> **Совет.** Настройте ежедневное копирование по расписанию (cron, systemd timer) и храните копии минимум 14 дней на другом сервере. Раз в квартал проверяйте восстановление на тестовом стенде.

10 Обновление

10.1 Обычное обновление

```
docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > before-update.dump
git fetch && git checkout vX.Y.Z
docker compose up -d --build
docker compose logs -f app
```

Миграции применятся при старте. Откат версии делается восстановлением резервной копии, сделанной перед обновлением.

10.2 Обновление до 0.2.0

Версия 0.2.0 добавляет структуру компании, профили доступа, вход по учётной записи домена и синхронизацию с Active Directory. Схему меняет миграция `0014_org_access`:

ТАБЛИЦА	ИЗМЕНЕНИЕ
`departments`	становится деревом: родитель, тип уровня, порядок, внешний идентификатор и источник; имена уникальны среди соседних, код необязателен
`department_heads`	новая: руководители подразделений с основным
`users`	основное подразделение, явный руководитель, «первое лицо», способ входа и связь с каталогом, профиль доступа, личные настройки, полномочия; пароль может быть пустым (доменная учётная запись, «без доступа»)
`user_departments` → `user_access_units`	первое по алфавиту подразделение пользователя становится основным, остальные — дополнительными подразделениями области доступа
`service_access`	уровень «нет доступа»; вместо признака «только свои подразделения» — область службы (`department_scope = true` → «Своя ветка»)
`access_profiles`, `app_settings`, `org_revision`	новые: профили доступа (встроенные создаются при старте), настройки структуры, ревизия структуры

Доступ существующих пользователей сохраняется: профиль им не назначается («Индивидуальная настройка» — вся компания, все классы данных, реестры и выгрузка, как в 0.1.0), уровни служб остаются прежними. Пароли и сессии не меняются: повторно входить не нужно. Подразделения 0.1.0 становятся верхним уровнем дерева — вложить их в нужные уровни можно после обновления.

Порядок (подробно — `docs/runbooks/upgrade-to-0.2.0.md`):

1. **Сделайте резервную копию** базы (раздел «Резервное копирование и восстановление»).
2. Получите версию и, если нужно, добавьте в `.env` новые переменные (`TIMEZONE`; вход по домену — позже):

   ```
   git fetch && git checkout v0.2.0
   docker compose up -d --build
   docker compose logs app | grep "Схема базы"     # Схема базы обновлена:
   0013_data_revisions → 0014_org_access
   ```

3. Проверьте `/api/health` (`"version": "0.2.0"`, `schema.current == schema.head`), войдите администратором, откройте «Пользователи» (у всех — «Индивидуальная настройка» и прежние службы), «Подразделения» и «Структуру компании».
4. Постройте структуру, назначьте профили доступа и при необходимости включите вход по домену («Руководство администратора»).

Откат. Миграция 0014 поддерживает откат, но лучший способ — восстановить резервную копию, сделанную перед обновлением, и вернуть прежнюю версию:

```
docker compose stop app
docker compose exec -T db pg_restore -U dashboards -d dashboards --clean --if-exists <
before-0.2.0.dump
git checkout v0.1.0 && docker compose up -d --build
```

Если после обновления уже внесены данные, которые нужно сохранить, откатите схему образом 0.2.0 (до переключения на 0.1.0):

```
docker compose stop app
docker compose run --rm --no-deps -w /app/backend --entrypoint alembic app downgrade
0013_data_revisions
git checkout v0.1.0 && docker compose up -d --build
```

При откате схемы теряется то, чего нет в 0.1.0: дерево (все подразделения становятся плоским списком, повторяющиеся названия получают номер, пустые коды — код вида `D12`), руководители подразделений, профили доступа и личные настройки, уровни «нет доступа», полномочия и привязка к домену. Доменные учётные записи и учётные записи «без доступа» после отката не могут войти — выдайте им пароль командой `reset-password`.

10.3 Обновление до 0.1.0 с версий до 0.1.0

Версия 0.1.0 — первая версия продукта; прежние версии («Dashboards API 1.1») хранили данные в SQLite в томе `db_data`. Миграция `0012_v0_1_0_schema` перестраивает все таблицы с переносом данных; дальше при том же старте применяются 0013 и 0014. Установки до 0.1.0 обновляются сразу до 0.2.0.

1. **Сделайте копию тома** со старой базой:

   ```
   docker run --rm -v dashboards_serv_db_data:/data -v "$PWD":/backup alpine cp
   /data/hr_dashboard.db /backup/hr_dashboard-before-0.1.0.db
   ```

2. **Вариант А — остаться на SQLite.** В `.env` задайте `DATABASE_URL=sqlite:////data/hr_dashboard.db` и подключите старый том к сервису `app` (раздел `volumes: - db_data:/data`). При старте база будет обновлена на месте.

3. **Вариант Б — перейти на PostgreSQL (рекомендуется).** Обновите копию старой базы, затем перенесите данные:

   ```
   # обновить копию SQLite до актуальной схемы
   docker compose run --rm --no-deps -w /app/backend --entrypoint python -v "$PWD":/backup -
   e DATABASE_URL=sqlite:////backup/hr_dashboard-before-0.1.0.db app -m app.cli migrate
   # создать схему в PostgreSQL и перенести данные
   docker compose up -d db
   docker compose run --rm -w /app/backend --entrypoint python app -m app.cli migrate
   docker compose run --rm -w /app/backend --entrypoint python -v "$PWD":/backup app
   scripts/sqlite_to_postgres.py /backup/hr_dashboard-before-0.1.0.db
   docker compose up -d
   ```

4. **Проверьте** после входа: месяцы службы персонала, недели отчёта ТП, число заявок Naumen и партнёрств на странице «О системе».

Что меняется для пользователей при обновлении с версий до 0.1.0:

- роли «Начальник службы персонала» и «Просмотр отдела» превращаются в доступ к службе персонала: «Управление» и «Просмотр» своей ветки структуры соответственно; роль «Просмотр» — в обычную учётную запись с её прежним доступом к службам;
- уровни доступа «чтение», «правка», «метрики», «админ» превращаются в «Просмотр», «Ввод данных», «Ввод данных», «Управление»;
- вход выполняется заново: прежние токены недействительны.

Подробности и проверки — `docs/runbooks/upgrade-to-0.1.0.md`.

11 Производительность и масштабирование

Конфигурация по умолчанию (4 ядра, `WEB_CONCURRENCY=9`, PostgreSQL в соседнем контейнере) рассчитана на 500–1000 одновременно работающих пользователей. Нагрузочная проверка 0.2.0: 1000 одновременных пользователей всех уровней доступа на 4 ядрах — p50 16 мс, p95 53 мс без ошибок сервера, структура компании (`/api/org/tree`) — p95 32 мс (`docs/performance/load-test-0.2.0.md`; методика и рекомендации по массовому входу — `load-test-0.1.0.md`); сценарий нагрузки — `loadtest/`.

Рекомендации:

- держите PostgreSQL на SSD; для больших объёмов заявок Naumen увеличьте `shared_buffers` до 25% памяти;
- при росте нагрузки сначала увеличивайте число процессов (`WEB_CONCURRENCY`), затем — число экземпляров приложения за балансировщиком; сессии хранятся в базе, «липкие» сессии не нужны;
- PDF-отчёты — самая тяжёлая операция (генерация графиков); при массовом формировании отчётов добавьте процессы;
- структура компании (`/api/org/tree`) строится один раз на ревизию структуры в каждом процессе: частые изменения структуры (например, синхронизация с AD каждые несколько минут) сбрасывают этот кэш — синхронизируйте раз в сутки или реже;
- проверка пароля в каталоге занимает время сети: держите `AUTH_LDAP_TIMEOUT` небольшим (5 с), чтобы недоступный каталог не задерживал вход надолго.

12 Обслуживание

Команды выполняются из каталога установки (`/opt/dashboards_serv`) на сервере.

ЗАДАЧА	КОМАНДА
Сбросить пароль пользователя	`docker compose exec -w /app/backend app python -m app.cli reset-password <логин>`
Создать администратора	`docker compose exec -w /app/backend -e ADMIN_PASSWORD='…' app python -m app.cli create-admin <логин>`
Удалить истёкшие сессии	`docker compose exec -w /app/backend app python -m app.cli cleanup-sessions`
Загрузить демо-данные в пустые разделы	`docker compose exec -w /app/backend app python -m app.cli demo-data`
Проверить схему базы	`docker compose exec -w /app/backend app python -m app.cli check-schema`
Применить миграции вручную	`docker compose exec -w /app/backend app python -m app.cli migrate`
План синхронизации с Active Directory	`docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad`
Синхронизировать с Active Directory	`docker compose exec -T -w /app/backend app python -m app.cli org-sync --ad --apply [--create-missing] [--deactivate-missing]`
Загрузить список сотрудников из файла	`docker compose cp список.csv app:/tmp/list.csv` , затем `docker compose exec -w /app/backend app python -m app.cli org-sync --csv /tmp/list.csv --apply --create-missing`
Резервная копия PostgreSQL	`docker compose exec -T db pg_dump -U dashboards -d dashboards -Fc > dashboards-$(date +%F).dump`

Истёкшие сессии не мешают работе (сервер их не принимает), но очистку раз в сутки удобно поставить в расписание вместе с резервным копированием и синхронизацией с AD.

13 Диагностика

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
Контейнер `app` перезапускается, в журнале «SECRET_KEY не задан»	Задайте `SECRET_KEY` не короче 32 символов в `.env`
`/api/health` отвечает `degraded`	Схема базы не актуальна: `docker compose exec -w /app/backend app python -m app.cli migrate`, смотрите журнал
«No module named app» при запуске `python -m app.cli`	Команда выполнена не в `/app/backend`: добавьте `-w /app/backend`
«Сессия устарела: обновите страницу»	Не совпал CSRF-токен (например, после выхода в другой вкладке) — обновите страницу
Вход сразу «выбрасывает» за HTTPS-прокси	Включён `SESSION_COOKIE_SECURE=true`, а сайт открыт по HTTP — откройте по HTTPS или выключите параметр
«Слишком много неудачных попыток»	Сработало ограничение частоты входа; подождите или перезапустите приложение
«Сервер домена не отвечает — попробуйте войти позже»	Каталог недоступен с сервера приложения: проверьте адрес, порт, межсетевой экран и сертификат на странице «Вход по домену» (**Проверить подключение**); локальные учётные записи входят по паролю системы
«Каталог не принял служебную учётную запись»	Неверные `AUTH_LDAP_BIND_DN` или `AUTH_LDAP_BIND_PASSWORD`, учётная запись заблокирована
Ошибка сертификата при подключении к каталогу	Укажите `AUTH_LDAP_CA_FILE` (сертификат центра сертификации домена) и смонтируйте файл в контейнер
«Учётной записи в системе нет» у сотрудника домена	Нет учётной записи с его e-mail: создайте её, загрузите сотрудников или включите `AUTH_LDAP_CREATE_USERS`
Сотрудник домена не находится или находится дважды	Проверьте `AUTH_LDAP_USER_FILTER` на странице «Вход по домену» (**Найти учётную запись**): нужна ровно одна запись
`org-sync` завершается с кодом 2	Каталог недоступен или не заданы `AD_SYNC_*` / `AUTH_LDAP_URL` и `AUTH_LDAP_BASE_DN`

ПРИЗНАК	ПРИЧИНА И РЕШЕНИЕ
`org-sync` завершается с кодом 1	В плане ошибки (например, цикл подчинения, повтор e-mail): запустите без `--apply` и с `--json`, исправьте данные в каталоге
Записи журнала за вечер попадают в следующий день, время в PDF сдвинуто	Задайте `TIMEZONE` часового пояса компании
Ошибка 413 при загрузке файла	Увеличьте `MAX_UPLOAD_MB` и `client_max_body_size` прокси
Медленные страницы при большом числе пользователей	Увеличьте `WEB_CONCURRENCY`, проверьте `max_connections` PostgreSQL и загрузку процессора

Журнал приложения: `docker compose logs app`. У каждого ответа API есть заголовок `X-Request-ID` — по нему запрос находится в журнале.